SOFTWARE TESTING PROJECT DOCUMENTATION

SMART LIBRARY CIRCULATION AND POLICY MANAGEMENT SYSTEM



AUTHORS:

- VIOLA ZENELI
- SIDRIT RRUSTEMI
- MEGI ALMADHI
- ARSILDO VELIU

Table of Contents

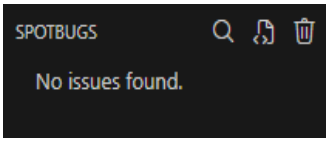
Part 1: Static Testing 2
Classes: 2
Interfaces: 7
Enumerations: 10
Part 2: Testing Analysis – Viola Zeneli 11
Part 2: Testing Analysis – Sidrit Rrustemi 14
Part 2: Testing Analysis – Megi Almadhi 17
Part 2: Testing Analysis – Arsildo Veliu 19
Part 3: Unit Testing 21
Part 3: Integration Testing 34
Part 3: System Testing 43
Part 1: GitHub Usage 47

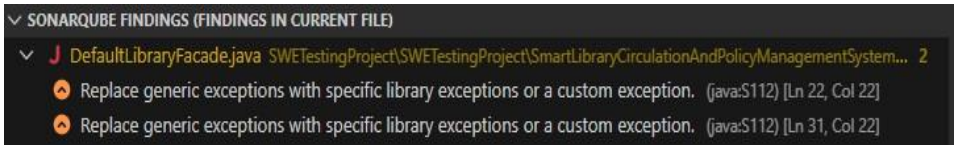
Part 1: Static Testing

Classes:

1. DefaultLibraryFacade:

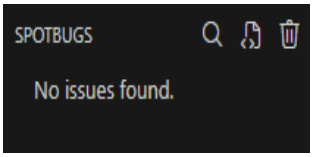
- **SonarQube** flags DefaultLibraryFacade because it uses overly generic exceptions such as Exception or Throwable in throws declarations and catch blocks. This makes it difficult for callers to understand what actually failed and can lead to important exceptions (for example, InterruptedException) being swallowed.
- To fix this, declare the specific exceptions that can occur (e.g., IOException, SQLException) or introduce a small domain-specific exception (such as LibraryFacadeException) and wrap the original exception as its cause. Avoid using catch (Exception); instead, catch only the exceptions you can meaningfully handle. If InterruptedException is caught, either rethrow it or restore the interrupt status using Thread.currentThread().interrupt().
- **SpotBugs** does not report an issue for this class, which is expected. SpotBugs focuses on detecting concrete bug patterns, whereas SonarQube also enforces design and maintainability rules such as S112.





2. BorrowRecord:

- **SonarQube** is issuing a warning because BorrowRecord overrides equals() but does not override hashCode(). In Java those two must be consistent: if equals() says two objects are equal, they must return the same hash code. If you leave hashCode() as the default (identity-based), equal objects can end up in different buckets in hash-based collections like HashSet or HashMap, which leads to lost entries, failed lookups, and subtle runtime bugs.
- To fix it, add a hashCode() that uses the same fields you use in equals(). Make sure you include exactly the same properties and be careful about using mutable fields if instances will be stored in hash-based collections. You can have your IDE generate it, use Objects.hash(field1, field2), or use a project utility like Lombok's @EqualsAndHashCode if that's allowed.
- **SpotBugs** did not flag any issues for this class.





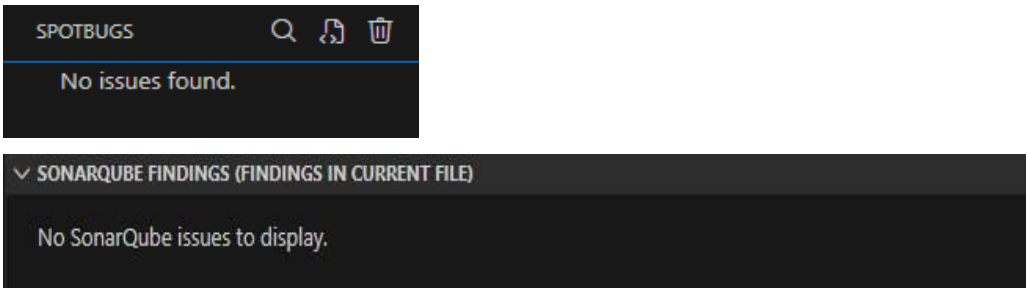
3. LibraryItem:

- **SonarQube** and **SpotBugs** didn't report any issues for LibraryItem class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.



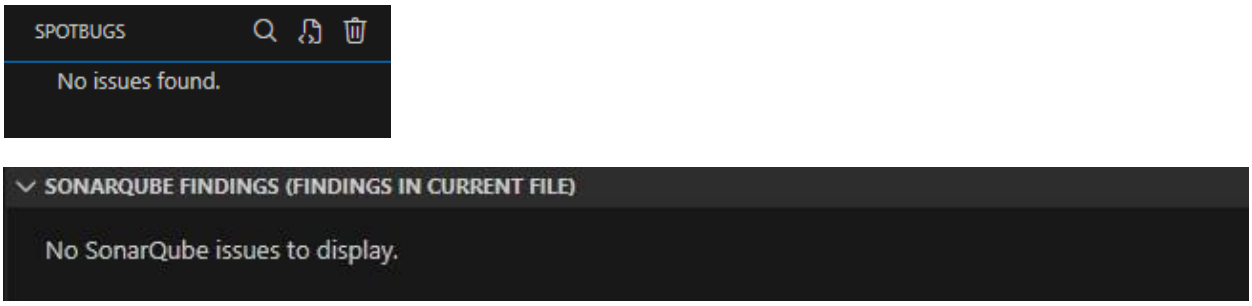
4. Member:

- **SonarQube** and **SpotBugs** didn't report any issues for Member class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.



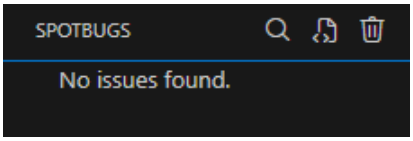
5. StubAlertService:

- **SonarQube** and **SpotBugs** didn't report any issues for StubAlertService class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.



6. DefaultBorrowPolicy:

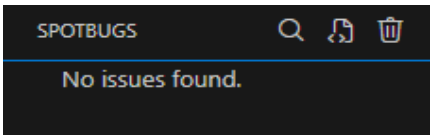
- **SonarQube** and **SpotBugs** didn't report any issues for DefaultBorrowPolicy class. Sonar raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.





7. DefaultPenaltyPolicy:

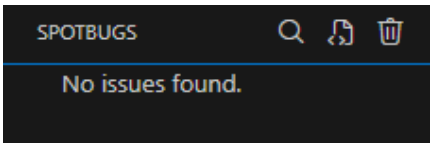
- **SonarQube** and **SpotBugs** didn't report any issues for DefaultPenaltyPolicy class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.

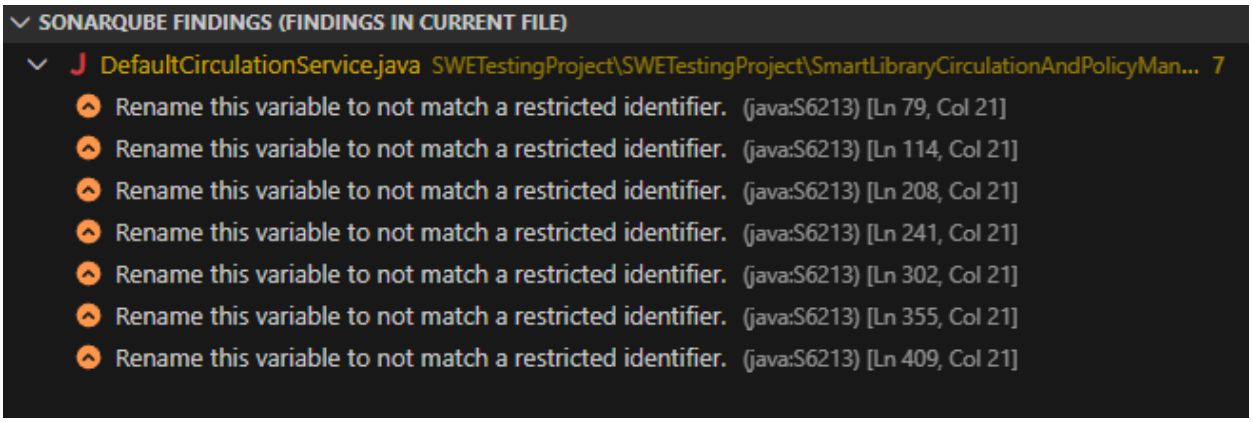




8. DefaultCirculationService:

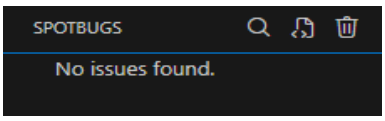
- **SonarQube** is issuing a warning because several local variables in DefaultCirculationService match a restricted identifier naming pattern. These names can be confusing or misleading, for example being too short, ambiguous, or resembling class, framework, or reserved identifiers.
- This can be fixed by renaming the affected variables to clear and descriptive camelCase names that reflect their purpose. Single-letter or ambiguous names should be avoided, and variable names should not reuse class or known framework identifiers. Renaming should be done using the IDE's refactoring tools to safely propagate the changes. If a variable name is intentionally chosen and properly documented, the rule may be suppressed for that specific line or adjusted in the SonarQube configuration, although renaming is the preferred solution.
- **SpotBugs** did not report any issues for this class.

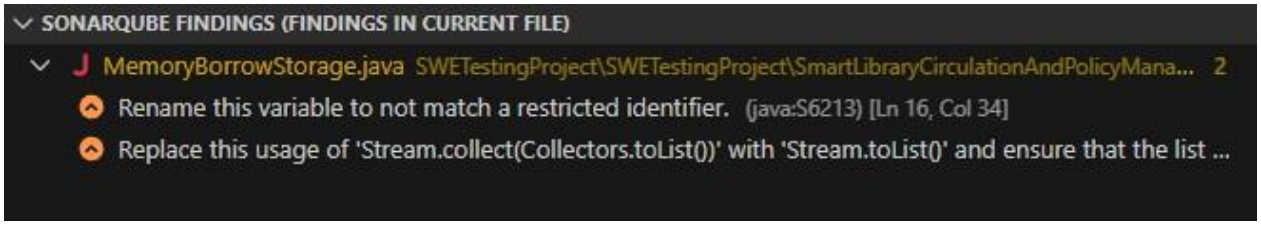




9. MemoryBorrowStorage:

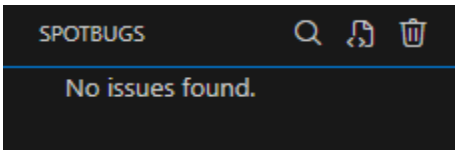
- **SonarQube** is issuing a warning because MemoryBorrowStorage contains two issues. First, rule **S6213** reports that some local variable names match a restricted identifier pattern, making them unclear or potentially misleading (for example, being too short, clashing with framework or common identifiers, or shadowing other names). Second, Sonar suggests replacing Stream.collect(Collectors.toList()) with Stream.toList().
- The naming issue can be fixed by renaming the affected variables to clear and descriptive camelCase names that reflect what the variable represents (for example, borrowList, borrowRecord, currentBorrowCount, index, or requestId). Renaming should be done using the IDE's refactoring tools to ensure consistency across the codebase.
- The stream-related issue can be fixed by using Stream.toList() if the project is running on Java 16 or newer and an unmodifiable list is acceptable. If a mutable list is required, Collectors.toList() should be kept, or the result of stream.toList() should be wrapped in a new ArrayList<>.
- **SpotBugs** did not report any issues for this class.





10. MemoryItemStorage:

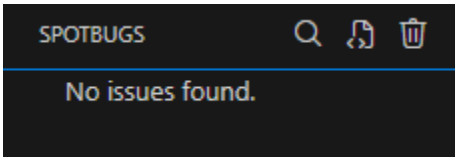
- **SonarQube** and **SpotBugs** didn't report any issues for MemoryItemStorage class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.





11. MemoryMemberStorage:

- **SonarQube** and **SpotBugs** didn't report any issues for MemoryMemberStorage class. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the class is flawless, only that these tools didn't flag anything.

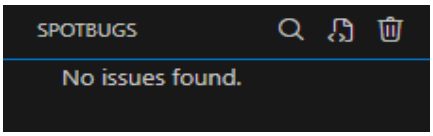




Interfaces:

1. ***LibraryFacade:***

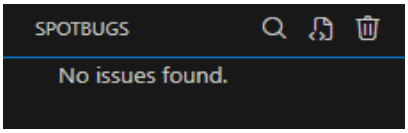
- **SonarQube** and **SpotBugs** didn't report any issues for LibraryFacade interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.





2. ***AlertService:***

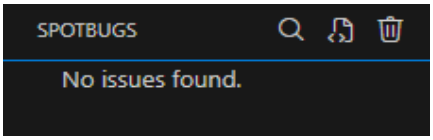
- **SonarQube** and **SpotBugs** didn't report any issues for AlertService interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.





3. ***BorrowPolicy:***

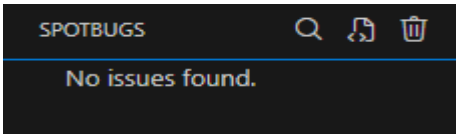
- **SonarQube** and **SpotBugs** didn't report any issues for BorrowPolicy interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.





4. ***PenaltyPolicy:***

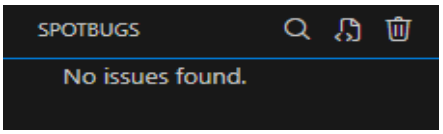
- **SonarQube** and **SpotBugs** didn't report any issues for PenaltyPolicy interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.





5. ***CirculationService:***

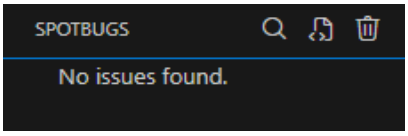
- **SonarQube** and **SpotBugs** didn't report any issues for CirculationService interface. Sonar raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.





6. ***ItemStorage:***

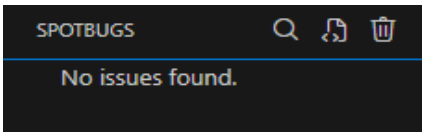
- **SonarQube** and **SpotBugs** didn't report any issues for ItemStorage interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.





7. ***MemberStorage:***

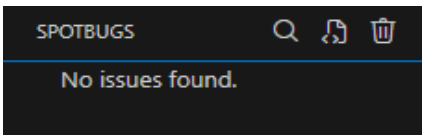
- **SonarQube** and **SpotBugs** didn't report any issues for MemberStorage interface. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the interface is flawless, only that these tools didn't flag anything.

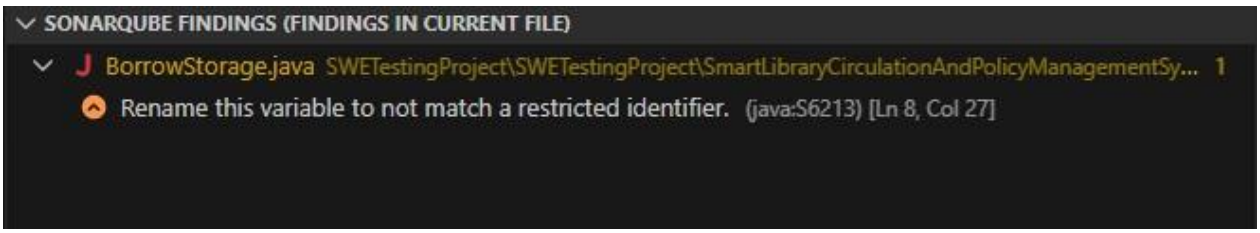




8. ***BorrowStorage:***

- **SonarQube** is issuing a warning because rule S6213 is raised in BorrowStorage, indicating that a local variable name matches a restricted identifier pattern. This means the variable name may be confusing or risky, for example being too short, shadowing important identifiers, or resembling reserved or framework names.
- This can be fixed by renaming the affected variable to a clear and descriptive camelCase name that reflects its purpose. Single-letter or ambiguous names should be avoided, and the IDE's rename refactoring tools should be used to ensure the change is applied safely throughout the code. If the variable name is intentionally chosen and properly documented, the rule may be suppressed for that specific line, although renaming remains the preferred solution.
- **SpotBugs** did not report any issues for this file.

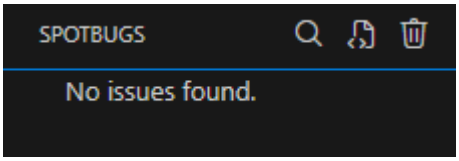




Enumerations:

1. ***MembershipType:***

- **SonarQube** and **SpotBugs** didn't report any issues for MembershipType enumeration. SonarQube raised no rules and SpotBugs found no bug patterns.
- That doesn't mean the enumeration is flawless, only that these tools didn't flag anything.





Part 2: Testing Analysis – Viola Zeneli

Boundary Value Testing

Method: DefaultCirculationService.calculateLateFeeWithDiscount(String itemCode, int discountPercent)

Boundary Value Testing is used to verify the behavior of the method at the **limits of valid input values**, where errors are most likely to occur.

In this method, BVT was applied only to **discountPercent** because it is the only input parameter with explicitly defined boundaries **[0,100]**.

I chose **Robust BVT**, because it checks **both valid and invalid boundaries**, so test cases cover values (**-1, 0, 1, 50, 99, 100, 101**).

TC	itemCode	discountPercent	Boundary Tested	Expected Result
BVT_1	"BOOK-001"	-1	Below min discount	IllegalArgumentException
BVT_2	"BOOK-001"	0	Min discount	Full fine returned
BVT_3	"BOOK-001"	1	Just above min	Fine reduced by 1%
BVT_4	"BOOK-001"	50	Nominal value	Fine reduced by 50%
BVT_5	"BOOK-001"	99	Just below max	Fine reduced by 99%
BVT_6	"BOOK-001"	100	Max discount	Late fee = 0
BVT_7	"BOOK-001"	101	Above max discount	IllegalArgumentException

Equivalence Class Testing

Method: DefaultCirculationService.assessItemReturnStatus(String itemCode, int daysLate)

Equivalence Class Testing is used to group input values that are expected to produce the **same output**, so only one representative value from each group needs to be tested.

For this method, equivalence classes are identified based on: the validity of itemCode, whether an active borrow exists and the range of daysLate.

Weak Robust Equivalence Class Testing was applied by selecting one representative input from each valid and invalid equivalence class without testing combinations of classes.

TC	itemCode	daysLate	Equivalence Class	Expected Result
EC_1	null	2	Invalid itemCode	IllegalArgumentException
EC_2	"BOOK-404"	2	No active borrow exists	"NO_ACTIVE_BORROW"
EC_3	"BOOK-001"	2	Already returned item	"ALREADY_RETURNED"
EC_4	"BOOK-001"	-1	Early return (daysLate<0)	"EARLY_RETURN"
EC_5	"BOOK-001"	0	On-time return (daysLate=0)	"ON_TIME"
EC_6	"BOOK-001"	2	Slightly late (1<=daysLate<=3)	"SLIGHTLY_LATE"
EC_7	"BOOK-001"	5	Moderately late (4<=daysLate<=7)	"MODERATELY_LATE"
EC_8	"BOOK-001"	10	Very late (8<=daysLate<=14)	"VERY_LATE"
EC_9	"BOOK-001"	20	Severely overdue (daysLate>14)	"SEVERELY_OVERDUE"

Code Coverage Testing

Method: DefaultCirculationService.canRenewBorrow(int memberId, String itemCode)

1. Statement Coverage

Statement coverage requires that every executable statement in the method is executed at least once, regardless of the outcome of decision points.

TC	memberId	itemCode	Path (where execution ends)	Expected
SC_1	-1	"BOOK-001"	Ends at **first if** (input validation)	false
SC_2	1	"BOOK-001"	Ends at **second if** (null check)	false
SC_3	1	"BOOK-001"	Ends at **final return**	true

2. Branch Coverage

Branch coverage requires that each decision (if) has both TRUE and FALSE outcomes executed at least once. It is about decision points, not individual boolean conditions.

TC	memberId	itemCode	Path (branches taken)	Expected
BC_1	-1	"BOOK-001"	First if = **true**	false
BC_2	1	"BOOK-001"	First if = **false**, Second if = **true**	false
BC_3	1	"BOOK-001"	First if = **false**, Second if = **false**	true

Naming Table of Boolean Conditions Used in Decision Logic of this Method

These condition names are used to avoid repeating full boolean expressions when defining condition coverage and MC/DC test cases later.

ID	Boolean Condition	Location in Code
C1	memberId <= 0	First if (input validation)
C2	itemCode == null	First if (input validation)
C3	itemCode.trim().isEmpty()	First if (input validation)
C4	member == null	Second if (existence check)
C5	item == null	Second if (existence check)
C6	record == null	Second if (existence check)
C7	record.getMemberId() == memberId	Final return (authorization check)
C8	!record.isReturned()	Final return (status check)
C9	!record.isOverdue(today)	Final return (time check)
C10	member.isActive()	Final return (account check)
C11	!borrowStorage.hasOverdueBorrows (memberId, today)	Final return (policy check)

3. Condition Coverage (Basic / Weak)

Condition coverage requires that each individual boolean condition in a decision evaluates to both TRUE and FALSE at least once, regardless of the overall decision outcome.

TC	memberId	itemCode	Forced Condition	Condition Value	Other Conditions	Expected Result
CC_1	-1	"BOOK-001"	C1	TRUE	others irrelevant	false
CC_2	1	null	C2	TRUE	others irrelevant	false
CC_3	1	" "	C3	TRUE	others irrelevant	false
CC_4	1	"BOOK-001"	C4	TRUE	C5,C6=FALSE	false
CC_5	1	"BOOK-001"	C5	TRUE	C4,C6=FALSE	false
CC_6	1	"BOOK-001"	C6	TRUE	C4,C5=FALSE	false
CC_7	1	"BOOK-001"	C7	FALSE	others TRUE	false
CC_8	1	"BOOK-001"	C8	FALSE	others TRUE	false
CC_9	1	"BOOK-001"	C9	FALSE	others TRUE	false
CC_10	1	"BOOK-001"	C10	FALSE	others TRUE	false
CC_11	1	"BOOK-001"	C11	FALSE	others TRUE	false
CC_12	1	"BOOK-001"	none	all TRUE	true	true

4. MC/DC Coverage

MC/DC (Modified Condition/Decision Coverage) requires that each condition is shown to independently affect the decision outcome while all other conditions are held constant.

MC/DC Scope (All decisions of this method)

Decision ID	Code Location	Decision Expression
D1	First if (input validation)	if (C1 || C2 || C3)
D2	Second if (existence check)	if (C4 || C5 || C6)
D3	Final return decision	C7 && C8 && C9 && C10 && C11

MC/DC Test Cases for D1, D2, D3 respectively

TC	C1	C2	C3	Decision Outcome	Return
D1_M1	F	F	F	FALSE	?
D1_M2	T	F	F	TRUE	FALSE
D1_M3	F	T	F	TRUE	FALSE
D1_M4	F	F	T	TRUE	FALSE

TC	C4	C5	C6	Decision Outcome	Return
D2_M1	F	F	F	FALSE	?
D2_M2	T	F	F	TRUE	FALSE
D2_M3	F	T	F	TRUE	FALSE
D2_M4	F	F	T	TRUE	FALSE

TC	C7	C8	C9	C10	C11	Decision Outcome	Return
D3_M1	T	T	T	T	T	TRUE	TRUE
D3_M2	F	T	T	T	T	FALSE	FALSE
D3_M3	T	F	T	T	T	FALSE	FALSE
D3_M4	T	T	F	T	T	FALSE	FALSE
D3_M5	T	T	T	F	T	FALSE	FALSE
D3_M6	T	T	T	T	F	FALSE	FALSE

#NOTE: If you go to the next table, the previous tables conditions are all false.

Part 2: Testing Analysis – Sidrit Rrustemi

1. Boundary Value Testing

Method tested: DefaultCirculationService.shouldSendReminder(String itemCode, int daysBeforeDue)

In this method, the critical numeric boundary is the daysBeforeDue parameter, which determines whether a reminder should be sent based on the item's due date. Additionally, the derived daysUntilDue (calculated from BorrowRecord.getDueDate()) affects the logic for sending reminders when items are overdue or due today.

I applied **Robust Boundary Value Testing (BVT)** by testing minimum and maximum boundaries (daysBeforeDue = 0 and 7), values just outside the valid range (-1 and 8), typical values (1, 3 and 6), and critical derived boundaries for daysUntilDue (-1, 0, and matching positive values).

TC	daysBeforeDue	daysUntilDue	Expected Result	Notes
BVT_1	-1	3	FALSE	Below lower boundary for daysBeforeDue (invalid input)
BVT_2	0	0	TRUE	Lower boundary matches due today
BVT_3	0	1	FALSE	Lower boundary, not due today
BVT_4	1	1	TRUE	Just above minimum matches due date
BVT_5	3	-1	TRUE	Overdue item, daysUntilDue < 0 triggers reminder
BVT_6	3	3	TRUE	Typical value, exact match
BVT_7	6	6	TRUE	Just below maximum matches due date
BVT_8	7	7	TRUE	Upper boundary matches due date
BVT_9	8	7	FALSE	Above upper boundary for daysBeforeDue (invalid input)I don't

2. Equivalence Class Testing

Method tested: DefaultCirculationService.validateMemberStatus(int memberId)

In this method, I identified equivalence classes based on three key factors: (1) memberId validity, (2) member existence in storage, and (3) combinations of member state attributes including active status, overdue item status, and loan capacity status.

Weak Robust Equivalence Class Testing was used - testing one representative from each valid and invalid class independently.

TC	memberId	Equivalence Class	Expected Result
EC_1	0	Invalid ID (memberId ≤ 0)	"INVALID_MEMBER_ID"
EC_2	1	Member does not exist in storage	"MEMBER_NOT_FOUND"
EC_3	2	All three issues: inactive, has overdue items, at max capacity	"SUSPENDED_MULTIPLE_ISSUES"
EC_4	3	Two issues: inactive and has overdue items	"SUSPENDED_INACTIVE_OVERDUE"
EC_5	4	Two issues: inactive and at max capacity	"SUSPENDED_INACTIVE_CAPACITY"
EC_6	5	Two issues: has overdue items and at max capacity (but active)	"RESTRICTED_OVERDUE_CAPACITY"
EC_7	6	Single issue: inactive member only	"INACTIVE"
EC_8	7	Single issue: has overdue items only (active member)	"OVERDUE_ITEMS"
EC_9	8	Single issue: at max capacity only (active member)	"AT_CAPACITY"
EC_10	9	No issues: active, no overdue items, below capacity	"ACTIVE_GOOD_STANDING"

3. Code Coverage Testing

Method tested: DefaultBorrowPolicy.canBorrow(Member member, int activeBorrows, boolean hasOverdueBooks)

3.1. Statement Coverage

Statement coverage ensures every executable line of code runs at least once during testing.

TC	Member Setup	activeBorrows	hasOverdueBooks	Result
SC_1	valid, active, maxLoans=5	2	FALSE	TRUE

3.2. Branch Coverage

Branch coverage requires testing each decision outcome - both the true and false branches of the return expression must be executed.

TC	Member	activeBorrows	hasOverdueBooks	Result
BC_1	valid, active, maxLoans=5	2	FALSE	TRUE
BC_2	null	2	FALSE	FALSE

3.3. Condition Coverage

Condition coverage mandates that every individual boolean condition evaluates to both true and false at least once across all test cases, independent of overall decision outcomes.

TC	Target Condition	Condition Value	Other Conditions	Result
CC_1	C1	TRUE	C2=T, C3=T, C4=T	TRUE
CC_2	C1	FALSE	not evaluated (short-circuit)	FALSE
CC_3	C2	TRUE	C1=T, C3=T, C4=T	TRUE
CC_4	C2	FALSE	C1=T, C3=T, C4=T	FALSE
CC_5	C3	TRUE	C1=T, C2=T, C4=T	TRUE
CC_6	C3	FALSE	C1=T, C2=T, C4=T	FALSE
CC_7	C4	TRUE	C1=T, C2=T, C3=T	TRUE
CC_8	C4	FALSE	C1=T, C2=T, C3=T	FALSE

3.4. MC/DC Coverage

Modified Condition/Decision Coverage (MC/DC) demonstrates that each condition independently influences the decision outcome. For each condition, two tests must exist where only that condition changes value while others remain fixed, and this change must alter the decision result.

TC	C1	C2	C3	C4	Decision Outcome & Result
DC_1	T	T	T	T	TRUE
DC_2	F	T	T	T	FALSE
DC_3	T	F	T	T	FALSE
DC_4	T	T	F	T	FALSE
DC_5	T	T	T	F	FALSE

Boolean Condition Naming Table used above

ID	Boolean Condition
C1	member != null
C2	member.isActive()
C3	activeBorrows < member.getMembershipType().getMaxLoans()
C4	!hasOverdueBooks

Part 2: Testing Analysis – Megi Almadhi

1. Boundary Value Testing (BVT)

Method: getRemainingBorrowCapacity(int memberId)- Calculates remaining borrow capacity.

Purpose: Test values at input domain edges where errors are most likely. The boundary is maximum loans (N = 5).

Formula: Math.max(0, maxLoans - activeBorrows)

Test Case	Active Borrows	Expected	Rationale
Below (N-1)	4	1	One below limit
At (N)	5	0	At maximum
Above (N+1)	6	0	Math.max prevents negatives

Key Finding: Above boundary validates Math.max prevents negative returns when limits are exceeded.

2. Equivalence Class Testing

Method: getFineCategory(String itemCode)- Categorizes fines by overdue days.

Purpose: Partition inputs into classes where all values produce identical behavior, reducing redundant tests.

Class	Condition	Test Values	Expected Output
1. Invalid	null/empty/whitespace	null, "", " "	IllegalArgumentException
2. No Borrow	No record exists	-	IllegalStateException
3. Returned	Already returned	-	NO_FINE_RETURNED
4. On Time	overdueDays ≤ 0	-5, 0	NO_FINE_ON_TIME

5. Low Fine	1 ≤ overdueDays ≤ 5	1, 3, 5	LOW_FINE
6. Medium Fine	6 ≤ overdueDays ≤ 14	6, 10, 14	MEDIUM_FINE
7. High Fine	overdueDays > 14	15, 100	HIGH_FINE

Strategy: Test representative values (minimum, typical, maximum) from each class for complete coverage.

3. Code Coverage Testing

Method: shouldSendReminder(String itemCode, int daysBeforeDue)- Determines if reminder is needed.

i. **Statement Coverage (100%):** All executable lines tested including validation, range checks, database lookups, calculations, and returns.

ii. **Branch Coverage (100%):** All decisions tested TRUE/FALSE:

- daysBeforeDue < 0, daysBeforeDue > 7
- record == null, record.isReturned()
- daysUntilDue == daysBeforeDue, daysUntilDue < 0

iii. **Condition Coverage (100%):** Each atomic condition tested independently in compound expressions:

- (daysBeforeDue < 0 || daysBeforeDue > 7) - both parts TRUE/FALSE
- (record == null || record.isReturned())- both parts TRUE/FALSE

iv. **MC/DC Coverage (100%):** Proves each condition independently affects outcome. Example: (daysBeforeDue < 0 || daysBeforeDue > 7)

- Base: Both FALSE (value=3) → FALSE
- First TRUE (value=-2) → TRUE (proves first matters)
- Second TRUE (value=10) → TRUE (proves second matters)

Result: 100% coverage across all criteria ensures thorough testing of logic paths and edge cases.

Part 2: Testing Analysis – Arsildo Veliu

1. Code Coverage Testing

Method Tested: canBorrowItemType()
This method checks whether a member can borrow an item based on valid member ID and item details, member active status, borrow capacity, overdue items, and membership-based value limits (Student: $100, Staff: $500, Guest: $50).

Statement Coverage
Statement coverage is achieved using a valid borrowing scenario where all conditions are satisfied. This test ensures every executable statement in the method is run at least once.

Branch Coverage
Branch coverage tests all decision points in both outcomes. The following validation cases are covered: invalid member ID (memberId <= 0), null item type, negative item value, and member not found in storage. Each case causes the method to return false, ensuring all validation branches are executed.

Condition Coverage
The final decision in the method is based on four conditions:
isActive && hasCapacity && noOverdue && valueLimitOk

Each condition is tested when it evaluates to false: inactive member, borrow limit reached, member has overdue items, and item value exceeds membership limit. A base test case confirms the method returns true when all conditions are true.

MC/DC (Modified Condition/Decision Coverage)
MC/DC is achieved by changing one condition at a time while keeping all others true:

- Base case: all conditions true → result true
- isActive false → result false
- hasCapacity false → result false
- noOverdue false → result false
- valueLimitOk false → result false

This demonstrates that each condition independently affects the result.

Additional Tests

Extra tests verify that item value limits work correctly for different membership types: staff members can borrow higher-value items, while guest members are restricted to lower-value items.

2. Equivalence Class Testing

Method Tested: getMembershipEligibility()

This method returns a membership status string based on number of current borrows, overdue status, and active status.

Identified Equivalence Classes

a. **Invalid borrow count** (currentBorrows < 0)
b. **Inactive membership** (regardless of other parameters)
c. Active membership with overdue items
d. **Excellent standing** (0 borrows, active, no overdue)
e. **Good standing** (1–2 borrows, active, no overdue)
f. **Moderate usage** (3–4 borrows, active, no overdue)
g. **Heavy usage** (5+ borrows, active, no overdue)

Representative and boundary values from each class are tested to ensure correct behaviour. For example, good standing tests both 1 borrow (minimum) and 2 borrows (maximum), while heavy usage tests 5 borrows (boundary), 10 (typical), and 100 (extreme).

3. Boundary Value Testing

Method Tested: getDaysUntilDue()

This method calculates how many days remain until an item is due.

Boundary Tested

The key boundary occurs when the due date equals the current date. Three boundary cases are tested:

a. Due yesterday → -1
b. Due today → 0
c. Due tomorrow → 1

Additional Boundary Cases

Tests for null item code, empty item code, no active borrow record, and returned item ensure the method handles invalid and edge cases correctly.

Summary

The testing approach combines coverage, equivalence class, and boundary value testing to validate the circulation service. Code coverage ensures all logic paths are executed, equivalence class testing efficiently covers input categories, and boundary value testing focuses on error-prone edge cases. Together, these techniques provide comprehensive test coverage for the Smart Library system.

Part 3: Unit Testing

Project Context

This section presents the **unit testing** carried out for the **Smart Library System**. The tests were designed and executed in an **ad-hoc** manner, focusing on verifying that each individual component works correctly in isolation, prior to integration with other components.

Purpose of Unit Testing

The main goals of unit testing in this project are to:

- Verify the correctness of individual classes and methods
- Detect defects at an early stage of development
- Ensure that each component behaves correctly for both valid and invalid inputs
- Improve code reliability and maintainability

Unit Testing Approach

Unit testing for the Smart Library System was performed by:

- Testing **one class at a time** in isolation
- Replacing dependencies with **mocks or stubs** where required
- Verifying both normal execution paths and exceptional cases
- Avoiding interactions with external components during testing

Test Organization and Responsibilities

To ensure clarity and consistency:

- Unit tests follow the **same package structure** as the main application (API, model, notification, policy, service, and storage)
- Each team member was assigned specific classes to test
- The assignment was based on the **number of test cases** of classes, so that the workload was approximately equal for all members
- Each team member implemented the unit tests for their assigned classes and completed the corresponding documentation tables

Unit Testing Documentation Tables

Class: DefaultLibraryUnitTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
constructor_throws_whenCirculationServiceNull()	Checks constructor validation so the facade cannot be created with a null service (prevents NullPointerException later).	Throws IllegalArgumentException with message "CirculationService cannot be null"	Pass
borrowFlow_callsRequestBorrow_onCirculationService()	Confirms borrowFlow() delegates correctly to CirculationService.requestBorrow() with the same parameters.	requestBorrow(1, "BOOK001") called exactly once	Pass
borrowFlow_throwsRuntimeException_whenCirculationServiceThrowsIllegalState()	Ensures system handles service failure (IllegalStateException) by wrapping it into a RuntimeException with a clear message.	Throws RuntimeException containing "Failed to complete borrow flow"	Pass
borrowFlow_throwsRuntimeException_whenCirculationServiceThrowsIllegalArgument()	Ensures invalid input/arguments from service (IllegalArgumentException) are wrapped into RuntimeException with a clear message.	Throws RuntimeException containing "Failed to complete borrow flow"	Pass
returnFlow_callsReturnItem_onCirculationService()	Confirms returnFlow() delegates correctly to CirculationService.returnItem() with the same item code.	returnItem("BOOK001") called exactly once	Pass
returnFlow_throwsRuntimeException_whenCirculationServiceThrowsIllegalState()	Ensures return flow wraps service errors into RuntimeException with clear message.	Throws RuntimeException containing "Failed to complete return flow"	Pass

Class: BorrowRecordUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
constructor_setsFields_andReturnedFalse()	Verify that the BorrowRecord constructor correctly initializes all fields and sets returned to false by default.	All fields set correctly; returned = false; borrow date not null; return date null	Pass
markReturned_setsReturnedTrue()	Verify that markReturned() correctly sets the returned status to true.	returned = true	Pass
markReturned_setsReturnDate()	Verify that markReturned() sets the return date to the current date.	return date equals today's date	Pass
isOverdue_returnsExpected_basedOnDaysAfterDue()	Verify that isOverdue() returns the correct overdue status based on the number of days after the due date.	Correct true/false value returned	Pass
overdueDays_returnsCorrectDays()	Verify that overdueDays() returns the correct number of overdue days when the item is not returned.	Correct overdue day count	Pass
overdueDays_returns0_whenReturned()	Verify that overdueDays() returns 0 when the item has already been returned.	0	Pass
constructor_throws_whenMemberIdInvalid()	Verify that the constructor throws an exception when the member ID is zero or negative.	IllegalArgumentException thrown	Pass
constructor_throws_whenItemCodeInvalid()	Verify that the constructor throws an exception when the item code is empty or blank.	IllegalArgumentException thrown	Pass

constructor_throws_whenItemCodeNull()	Verify that the constructor throws an exception when the item code is null.	IllegalArgumentException thrown	Pass
constructor_throws_whenDueDateNull()	Verify that the constructor throws an exception when the due date is null.	IllegalArgumentException thrown	Pass
equals_returnsTrue_forSameRecord()	Verify that equals() returns true when comparing the same BorrowRecord instance.	true	Pass
equals_returnsFalse_forDifferentRecord()	Verify that equals() returns false when comparing different BorrowRecord objects.	false	Pass

Class: LibraryItemUnitTest

Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
constructor_setsCodeAndAvailability()	Verify that the LibraryItem constructor correctly sets the item code and availability status.	Code = "BOOK001", available = true	Pass
setAvailable_updatesAvailability()	Verify that setAvailable() correctly updates the availability status when set to false and true.	Availability updates correctly	Pass

Class: MembershipTypeUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
membershipType_hasExpectedMaxLoansAndDays()	Verify that each membership type has the correct maximum loans and loan duration values.	Correct max loans and loan days for each type	Pass

Class: MemberUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
constructor_setsIdActiveMembershipType()	Verify that the Member constructor correctly sets the ID, active status, and membership type.	ID = 1, active = true, type = STUDENT	Pass

Class: StubAlertServiceUnitTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
sendAlert_storesLastMemberIdAndMessage()	Verifies the stub stores the last alert data so tests can inspect calls (memberId + message).	After first call: lastMemberId=1, lastMessage="Test message". After second call: lastMemberId=2, lastMessage="Another message"	Pass

Class: DefaultBorrowPolicyUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
canBorrow_returnsFalse_whenMemberIsNull()	Verify that canBorrow() returns false when the member is null.	false	Pass
canBorrow_returnsFalse_whenMemberInactive()	Verify that canBorrow() returns false when the member is inactive.	false	Pass
canBorrow_returnsFalse_whenActiveBorrowsEqualsMax()	Verify that canBorrow() returns false when active borrows equal the membership maximum.	false	Pass
canBorrow_returnsTrue_whenActiveBorrowsLessThanMax_andNoOverdue_andMemberActive()	Verify that canBorrow() returns true when the member is active, has no overdue items, and active borrows are below the limit.	true	Pass
canBorrow_returnsFalse_whenHasOverdueTrue()	Verify that canBorrow() returns false when the member has overdue items.	false	Pass

Class: DefaultPenaltyPolicyUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
calculatePenalty_returnsExpectedAmount()	Verify that calculatePenalty() returns the correct penalty amount based on overdue days, including boundary and capped values.	Correct penalty amount returned	Pass

Class: DefaultCirculationServiceUnitTest

Responsible Members: Viola Zeneli & Sidrit Rrustemi

Test Case	Test Description & Reason	Expected Output	Actual Result
Viola Zeneli			
requestBorrow_throws_whenMemberIdInvalid()	Ensures borrowing is rejected when member ID is zero or negative.	IllegalArgumentException thrown	IllegalArgumentException thrown
requestBorrow_throws_whenItemCodeInvalid()	Verifies borrowing fails when item code is null or empty.	IllegalArgumentException thrown	IllegalArgumentException thrown
requestBorrow_throws_whenMemberNotFound()	Confirms exception is thrown when the member does not exist.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_throws_whenItemNotFound()	Ensures exception when the item does not exist.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_throws_whenItemNotAvailable()	Verifies borrowing is blocked if item is unavailable.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_throws_whenBorrowPolicyRejects()	Confirms borrowing is denied when policy rules fail.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_updatesAvailabilityToFalse_andSavesBorrowRecord_whenValid()	Ensures availability is updated and record is saved.	Availability updated, record saved	Availability updated, record saved
requestBorrow_savesBorrowRecordWithDueDate_todayPlusMaxLoanDays()	Verifies due date calculation based on membership type.	Correct due date saved	Correct due date saved
returnItem_throws_whenItemCodeNull()	Ensures return fails when item code is invalid.	IllegalArgumentException thrown	IllegalArgumentException thrown
returnItem_throws_whenNoActiveBorrow()	Confirms exception when no active borrow exists.	IllegalStateException thrown	IllegalStateException thrown
returnItem_marksRecordReturned_andUpdatesAvailabilityTrue()	Verifies item is returned and availability restored.	Record returned, availability true	Record returned, availability true

returnItem_sendsAlert_whenRecordIsOverdue()	Ensures alert is sent for overdue return.	Alert sent	Alert sent
returnItem_doesNotSendAlert_whenRecordNotOverdue()	Confirms no alert for on-time return.	No alert sent	No alert sent
canBorrowMore_throws_whenMemberNotFound()	Ensures exception when member does not exist.	IllegalStateException thrown	IllegalStateException thrown
canBorrowMore_returnsTrue_whenActiveBorrowsLessThanMax()	Confirms borrowing allowed under capacity.	true	true
canBorrowMore_returnsFalse_whenActiveBorrowsEqualsMax()	Ensures borrowing denied at capacity.	false	false
canBorrowMore_returnsFalse_whenActiveBorrowsGreaterThanMax()	Confirms borrowing denied when capacity exceeded.	false	false
calculateFine_returns0_whenNotOverdue()	Verifies no fine for non-overdue items.	0	0
calculateFine_returnsCorrectAmount_whenOverdue()	Ensures fine is calculated correctly.	Correct fine returned	Correct fine returned
calculateFine_returns0_whenItemReturned()	Confirms no fine for returned items.	0	0
calculateFine_throws_whenNoActiveBorrow()	Ensures exception when no borrow exists.	IllegalStateException thrown	IllegalStateException thrown
calculateFine_throws_whenItemCodeNull()	Ensures invalid item code throws exception.	IllegalArgumentException thrown	IllegalArgumentException thrown
isEligibleToBorrow_returnsTrue_whenAllConditionsMet()	Confirms eligibility when all rules pass.	true	true
isEligibleToBorrow_returnsFalse_whenMemberNull()	Ensures borrowing denied if member missing.	false	false
isEligibleToBorrow_returnsFalse_whenItemNull()	Ensures borrowing denied if item missing.	false	false
isEligibleToBorrow_returnsFalse_whenMemberInactive()	Confirms inactive members cannot borrow.	false	false
isEligibleToBorrow_returnsFalse_whenItemNotAvailable()	Ensures unavailable items cannot be borrowed.	false	false
isEligibleToBorrow_returnsFalse_whenAtCapacity()	Confirms borrowing denied at capacity.	false	false
isEligibleToBorrow_returnsFalse_whenHasOverdue()	Ensures overdue members cannot borrow.	false	false
isEligibleToBorrow_returnsFalse_whenMemberIdInvalid()	Ensures invalid member ID returns false.	false	false

isEligibleToBorrow_returnsFalse_whenItemCodeNull()	Ensures null item code returns false.	false	false
getRemainingBorrowCapacity_returnsCorrectValue()	Verifies correct remaining capacity calculation.	Correct capacity returned	Correct capacity returned
getRemainingBorrowCapacity_throws_whenMemberNotFound()	Ensures exception when member missing.	IllegalStateException thrown	IllegalStateException thrown
validateMemberStatus_returnsExpectedStatus()	Confirms correct status for different member states.	Correct status returned	Correct status returned
validateMemberStatus_returnsInvalid_whenMemberIdNegativeOrZero()	Ensures invalid IDs return error status.	INVALID_MEMBER_ID	INVALID_MEMBER_ID
validateMemberStatus_returnsMEMBER_NOT_FOUND_whenMemberNull()	Confirms MEMBER_NOT_FOUND status.	MEMBER_NOT_FOUND	MEMBER_NOT_FOUND
Sidrit Rrustemi			
getFineCategory_returnsExpectedCategory()	Tests fine categorization based on overdue days.	Correct category (NO_FINE, LOW, MEDIUM, HIGH)	Correct category
getFineCategory_returnsNO_FINE_RETURNED_whenAlreadyReturned()	Verifies no fine category for returned items.	NO_FINE_RETURNED	NO_FINE_RETURNED
getFineCategory_throws_whenNoActiveBorrow()	Ensures exception when no active borrow exists.	IllegalStateException thrown	IllegalStateException thrown
canRenewBorrow_returnsTrue_whenAllConditionsMet()	Confirms renewal allowed when all conditions satisfied.	true	true
canRenewBorrow_returnsFalse_whenMemberNotFound()	Verifies renewal denied when member doesn't exist.	false	false
canRenewBorrow_returnsFalse_whenItemNotFound()	Ensures renewal denied when item doesn't exist.	false	false
canRenewBorrow_returnsFalse_whenNoActiveBorrow()	Confirms renewal denied with no active borrow.	false	false
canRenewBorrow_returnsFalse_whenNotTheBorrower()	Verifies renewal denied for non-borrower.	false	false

canRenewBorrow_returnsFalse_whenAlreadyReturned()	Ensures renewal denied for returned items.	false	false
canRenewBorrow_returnsFalse_whenOverdue()	Confirms renewal denied for overdue items.	false	false
canRenewBorrow_returnsFalse_whenMemberInactive()	Verifies renewal denied for inactive members.	false	false
canRenewBorrow_returnsFalse_whenMemberHasOtherOverdueItems()	Ensures renewal denied when member has other overdue items.	false	false
getTotalOverdueDays_returns0_whenNoOverdueItems()	Confirms zero when no items are overdue.	0	0
getTotalOverdueDays_returnsSumOfOverdueDays_whenMultipleOverdueItems()	Verifies correct sum of overdue days across items.	Sum of overdue days	Sum of overdue days
getTotalOverdueDays_ignoresReturnedItems()	Ensures returned items excluded from calculation.	Overdue days excluding returned	Overdue days excluding returned
getTotalOverdueDays_throws_whenMemberIdInvalid()	Verifies exception for invalid member ID.	IllegalArgumentException thrown	IllegalArgumentException thrown
getDaysUntilDue_returnsCorrectDays()	Tests correct calculation of days until due date.	Correct day count	Correct day count
getDaysUntilDue_returns0_whenAlreadyReturned()	Confirms zero for returned items.	0	0
getDaysUntilDue_throws_whenNoActiveBorrow()	Ensures exception when no active borrow exists.	IllegalStateException thrown	IllegalStateException thrown
getDaysUntilDue_throws_whenItemCodeNull()	Verifies exception for null item code.	IllegalArgumentException thrown	IllegalArgumentException thrown
calculateLateFeeWithDiscount_appliesDiscountCorrectly()	Tests discount application to late fees.	Discounted fee amount	Discounted fee amount
calculateLateFeeWithDiscount_throws_whenDiscountInvalid()	Ensures exception for invalid discount percentage.	IllegalArgumentException thrown	IllegalArgumentException thrown
getMembershipEligibility_returnsExpectedStatus()	Tests eligibility status for various scenarios.	Expected eligibility status	Expected eligibility status
assessItemReturnStatus_returnsExpectedStatus()	Verifies return status based on days late.	Correct status (EARLY, ON_TIME, LATE, etc.)	Correct status

assessItemReturnStatus_returnsNO_ACTIVE_BORROW_whenNoRecord()	Confirms status when no borrow record exists.	NO_ACTIVE_BORROW	NO_ACTIVE_BORROW
assessItemReturnStatus_returnsALREADY_RETURNED_whenReturned()	Ensures status for already returned items.	ALREADY_RETURNED	ALREADY_RETURNED
canBorrowItemType_respectsValueLimits()	Tests value limits for different membership types.	Correct boolean based on limits	Correct boolean
canBorrowItemType_returnsFalse_whenMemberIdInvalid()	Verifies false for invalid member ID.	false	false
canBorrowItemType_returnsFalse_whenItemTypeNull()	Ensures false for null item type.	false	false
canBorrowItemType_returnsFalse_whenItemValueNegative()	Confirms false for negative item value.	false	false
canBorrowItemType_returnsFalse_whenMemberNotFound()	Verifies false when member doesn't exist.	false	false
canBorrowItemType_returnsFalse_whenMemberInactive()	Ensures false for inactive members.	false	false
canBorrowItemType_returnsFalse_whenNoCapacity()	Confirms false when member at capacity.	false	false
canBorrowItemType_returnsFalse_whenHasOverdue()	Verifies false when member has overdue items.	false	false
shouldSendReminder_returnsExpected()	Tests reminder logic based on days until due and threshold.	Correct boolean	Correct boolean
shouldSendReminder_returnsFalse_whenDaysBeforeDueOutOfRange()	Ensures false for out-of-range reminder days.	false	false
shouldSendReminder_returnsFalse_whenNoActiveBorrow()	Confirms false when no active borrow exists.	false	false
shouldSendReminder_returnsFalse_whenAlreadyReturned()	Verifies false for already returned items.	false	false

Class: MemoryBorrowStorageUnitTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
save_shouldStoreRecord()	Checks save() stores a borrow record and is counted as active.	findActiveBorrow("BOOK001") not null and countActiveBorrows(1) =1	Pass
findActiveBorrow_shouldReturnMatchingNotReturnedRecord()	Confirms active borrow record is returned for item that is borrowed and not returned.	Returned record not null, itemCode="BOOK001", returned=false	Pass
findActiveBorrow_shouldReturnNullWhenNoMatchingRecord()	Ensures method returns null when no record exists for that item.	findActiveBorrow("NONEXISTENT") returns null	Pass
findActiveBorrow_shouldIgnoreReturnedRecords()	Confirms returned records are not treated as active.	After markReturned(), findActiveBorrow("BOOK001") returns null	Pass
countActiveBorrows_shouldCountCorrectly()	Verifies correct count per member using multiple inputs (parameterized).	Member 1 → 2, Member 2 → 1, Member 3 → 0	Pass
countActiveBorrows_shouldReturnZeroWhenNone()	Ensures 0 is returned when a member has no active borrows.	countActiveBorrows(999) returns 0	Pass
hasOverdueBorrows_shouldReturnTrueWhenMemberHasActiveOverdueBorrow()	Confirms overdue logic works for past due date and still-active borrow.	hasOverdueBorrows(1, today) returns true	Pass
hasOverdueBorrows_shouldReturnFalseWhenMemberHasNoOverdueBorrow()	Confirms future due date is not considered overdue.	hasOverdueBorrows(1, today) returns false	Pass
hasOverdueBorrows_shouldIgnoreReturnedOverdueBorrow()	Ensures overdue records that are already returned are ignored.	After markReturned(), hasOverdueBorrows(1, today) returns false	Pass
findAll_shouldReturnAllSavedRecords()	Confirms retrieving all records returns correct amount.	findAll() list size = 3	Pass
findByMember_returnsOnlyMemberRecords()	Ensures member filtering returns only that member's records.	For member 1: size = 2 and all records have memberId=1	Pass
findByMember_returnsEmptyList_whenNoRecords()	Ensures method returns an empty list (not null) when no records exist for member.	Returned list is not null and is empty	Pass

Class: MemoryItemStorageUnitTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
save_shouldStoreItem()	Ensures calling save() stores the item and it can be retrieved later.	find("BOOK001") returns the same stored LibraryItem object	Pass
find_shouldReturnItemWhenExists()	Confirms find() returns correct item details when item exists in storage.	Returned item is not null, code="BOOK001", available=true	Pass
find_shouldReturnNullWhenNotExists()	Ensures find() returns null when item code is missing (expected behavior).	find("NONEXISTENT") returns null	Pass
updateAvailability_shouldChangeAvailabilityWhenItemExists()	Confirms updating availability changes stored item state correctly.	After update false → item available=false; after update true → item available=true	Pass
updateAvailability_shouldDoNothingWhenItemNotExists()	Ensures method handles missing items safely (no crash).	No exception thrown when updating availability for missing item	Pass

Class: MemoryMemberStorageUnitTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
save_shouldStoreMember()	Verify that save() correctly stores a member in memory storage.	Member stored successfully	Pass
find_shouldReturnMemberWhenExists()	Verify that find() returns the correct member when the member exists in storage.	Correct member returned	Pass
find_shouldReturnNullWhenNotExists()	Verify that find() returns null when the member does not exist in storage.	null	Pass

Part 3: Integration Testing

Project Context

This section presents the **integration testing activities** carried out for the **Smart Library System**. Integration testing is used to verify that **multiple components work correctly together**, after their individual correctness has been confirmed through unit testing.

Purpose of Integration Testing

The main goals of integration testing in this project are to:

- Verify the correct interaction between integrated components
- Ensure data flows correctly across different system layers
- Detect interface and interaction defects between modules
- Validate that combined components behave as expected in real scenarios

Integration Testing Approach

Integration testing for the Smart Library System was performed using a **bottom-up strategy**, which involves:

- Starting from low-level components and gradually integrating higher-level modules
- Testing real interactions between components without mocking previously tested units
- Ensuring each integration level is validated before moving to the next one
- Building confidence incrementally as the system grows in complexity

The integration levels were structured as follows, where each higher level assumes that all lower levels have already been successfully tested :

- **Level A:** Memory storage integration
- **Level B:** Policy integration
- **Level C:** CirculationService integration
- **Level D:** LibraryFacade integration

Test Organization and Responsibilities

To ensure clarity and consistency:

- Integration tests follow a clear **layered structure** aligned with the system architecture
- Each team member was assigned responsibility for specific integration levels or classes
- The assignment was based on the **number and complexity of test cases**, ensuring a balanced workload
- Each team member implemented the integration tests for their assigned components and completed the corresponding documentation tables

Integration Testing Documentation Tables

Class: A_MemoryStoragesIntegrationTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
saveMember_thenFind_returnsSameMember()	Verifies that a saved member can be retrieved correctly from memory storage	Member object returned with correct ID, status, and membership type	Member retrieved successfully
saveItem_thenFind_returnsSameItem()	Ensures library items are saved and fetched correctly	Item returned with correct code and availability	Item retrieved successfully
updateAvailability_changesItemAvailability()	Confirms item availability updates correctly	Item availability becomes false	Availability updated successfully
saveBorrowRecord_thenFindActiveBorrow_returnsRecord()	Checks active borrow retrieval after saving	Active borrow record returned	Record retrieved successfully
findActiveBorrow_whenReturned_returnsNull()	Ensures returned borrows are not considered active	Null returned	Null returned correctly
countActiveBorrows_countsOnlyNotReturned()	Verifies counting of active (non-returned) borrows	Count equals number of active borrows	Correct count returned
hasOverdueBorrows_whenAnyOverdueAndNotReturned_returnsTrue()	Detects overdue borrow records	True returned	Overdue detected
findAll_returnsAllSavedRecords()	Ensures all borrow records are returned	List size equals number saved	All records returned

Class: B_PolicyIntegrationTest
Responsible Member: Megi Almadhi

Test Case	Test Description & Reason	Expected Output	Actual Result
borrowPolicy_withRealMember_rejectsInactive()	Prevents inactive members from borrowing	Borrow denied	Borrow denied
borrowPolicy_withRealMember_rejectsAtCapacity()	Prevents borrowing when limit reached	Borrow denied	Borrow denied
borrowPolicy_withRealMember_rejectsWhenOverdue()	Prevents borrowing when overdue exists	Borrow denied	Borrow denied
borrowPolicy_withRealMember_allowsWhenAllGood()	Allows borrowing when all rules satisfied	Borrow allowed	Borrow allowed
penaltyPolicy_calculatesCorrectFine_for1DayOverdue()	Calculates fine for 1 overdue day	$1.00 fine	Correct fine
penaltyPolicy_calculatesCorrectFine_for5DaysOverdue()	Calculates fine within grace period	$5.00 fine	Correct fine
penaltyPolicy_calculatesCorrectFine_for6DaysOverdue()	Applies higher rate after grace period	$12.00 fine	Correct fine
penaltyPolicy_capsAt100_for50PlusDaysOverdue()	Ensures fine does not exceed cap	$100.00 max	Cap applied correctly

Class: C_CirculationServiceIntegrationTest
Responsible Members: Viola Zeneli & Sidrit Rrustemi

Test Case	Test Description & Reason	Expected Output	Actual Result
Viola Zeneli			
requestBorrow_happyPath_savesRecord_andMarksItemUnavailable()	Verifies that a valid borrow request correctly saves a borrow record and marks the item as unavailable.	Borrow record saved and item availability set to false	Borrow record saved and item marked unavailable
requestBorrow_whenMemberNotFound_throwsIllegalStateException()	Ensures borrowing fails when the member does not exist in storage.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenItemNotFound_throwsIllegalStateException()	Verifies borrowing fails when the requested item does not exist.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenItemNotAvailable_throwsIllegalStateException()	Confirms borrowing is blocked when the item is already unavailable.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenBorrowPolicyRejects_dueToInactiveMember_throwsIllegalStateException()	Ensures inactive members are prevented from borrowing items.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenBorrowPolicyRejects_dueToLimitReached_throwsIllegalStateException()	Verifies borrowing is denied when the member has reached the maximum loan limit.	IllegalStateException thrown	IllegalStateException thrown
requestBorrow_whenBorrowPolicyRejects_dueToOverdue_throwsIllegalStateException()	Confirms borrowing is denied when the member has overdue items.	IllegalStateException thrown	IllegalStateException thrown

returnItem_happyPath_marksReturned_andMarksItemAvailable()	Verifies that returning an item marks the borrow as returned and restores item availability.	Record marked returned and item available	Record returned and item available
returnItem_whenOverdue_sendsAlert()	Ensures an alert is sent when an overdue item is returned.	Alert sent to member	Alert sent
returnItem_whenNotOverdue_doesNotSendAlert()	Confirms no alert is sent for items returned on time.	No alert sent	No alert sent
canBorrowMore_whenUnderLimit_returnsTrue()	Verifies the service allows borrowing when the member is under the loan limit.	true	true
canBorrowMore_whenAtLimit_returnsFalse()	Ensures borrowing is not allowed when the member has reached the loan limit.	false	false
multipleMembersCanBorrowDifferentItems_simultaneously()	Verifies that multiple members can borrow different items at the same time.	Both borrow operations succeed	Both borrows succeed
sameMember_cannotBorrowSameItemTwice()	Confirms a member cannot borrow the same item more than once simultaneously.	IllegalStateException thrown	IllegalStateException thrown
borrowReturnBorrowAgain_sameItem_worksCorrectly()	Ensures an item can be borrowed again after it has been returned.	Second borrow succeeds	Borrowing and returning the item functions correctly.
borrowMultipleItems_respectsCapacity()	Verifies borrowing multiple items respects membership capacity limits.	IllegalStateException thrown when limit exceeded	IllegalStateException thrown

Sidrit Rrustemi			
calculateFine_usesRealPenaltyPolicy_withOverdueRecord()	Verifies that fine calculation uses the real penalty policy for overdue items	Fine of $3.0 for 3 days overdue	Fine calculated as $3.0
calculateFine_returns0_whenNotOverdue()	Verifies that no fine is charged when item is not overdue	Fine of $0.0	Fine returned as $0.0
getRemainingBorrowCapacity_decreasesAfterBorrow_increasesAfterReturn()	Verifies that borrow capacity decreases after borrowing and increases after returning	Capacity: 3 → 2 → 3	Capacity changes correctly through lifecycle: 3 → 2 → 3
validateMemberStatus_reflectsRealStorageState()	Verifies that member status validation reflects actual storage state including overdue items	Status changes from ACTIVE_GOOD_STANDING to OVERDUE_ITEMS	Status reflects real-time storage state
canRenewBorrow_checksRealOverdueStatus()	Verifies that renewal eligibility checks actual overdue status from storage	Returns true when no overdue, false when overdue items exist	Renewal eligibility correctly based on overdue status: true→false as expected
getFineCategory_correctForDifferentOverdueLevels()	Verifies that fine categories are correctly assigned based on overdue duration	NO_FINE_ON_TIME, LOW_FINE, MEDIUM_FINE categories	Categories assigned correctly per overdue level
getDaysUntilDue_countdownToZero_thenNegative()	Verifies that days until due counts down and becomes negative when overdue	Positive for future dates, negative for past dates	Days calculated correctly including negative values: Days: +3, -2
calculateLateFeeWithDiscount_appliesCorrectly()	Verifies that discount percentage is correctly applied to calculated fines	$20.0 fine with 50% discount = $10.0	Discount applied correctly to fine: $20.0→$10.0

getMembershipEligibility_basedOnBorrowHistory()	Verifies that membership eligibility is determined by borrow count and overdue status	Different eligibility levels based on usage patterns	Eligibility categories assigned correctly
assessItemReturnStatus_progressesThroughCategories()	Verifies that return status categories progress correctly based on days overdue	ON_TIME → SLIGHTLY_LATE → MODERATELY_LATE → VERY_LATE → SEVERELY_OVERDUE	Status categories progress correctly
canBorrowItemType_respectsValueLimits()	Verifies that item value limits are enforced based on membership type	STUDENT max $100, GUEST max $50 enforced	Value limits respected per membership type
shouldSendReminder_triggersAtRightTime()	Verifies that reminders are triggered at appropriate times before/after due date	Reminders sent at threshold days and for overdue items	Reminder triggers work correctly
memberBecomesInactive_cannotBorrowMore()	Verifies that inactive members cannot borrow even if previously active	Exception thrown when inactive member attempts to borrow	Inactive status prevents borrowing: IllegalStateException thrown
itemBecomesUnavailable_afterFirstBorrow()	Verifies that borrowed items cannot be borrowed by other members	Exception thrown when attempting to borrow unavailable item	Unavailable items cannot be borrowed: IllegalStateException thrown
isEligibleToBorrow_checksAllConditions_withRealData()	Verifies that eligibility checks consider all conditions including capacity limits	True when eligible, false when at capacity	All eligibility conditions checked correctly: true→false at capacity
getTotalOverdueDays_sumsAcrossMultipleBorrows()	Verifies that overdue days are correctly summed across multiple active borrows	Total of 8 days (3 + 5, ignoring non-overdue)	Overdue days summed correctly: Total = 8 days

Class: D_LibraryFacadeIntegrationTest
Responsible Member: Arsildo Veliu

Test Case	Test Description & Reason	Expected Output	Actual Result
borrowFlow_delegatesToCirculationService_andBorrowIsProcessed()	Verifies borrow flow updates item and borrow record	Item unavailable, borrow created	Borrow successful
returnFlow_delegatesToCirculationService_andReturnIsProcessed()	Verifies return updates item and borrow record	Item available, borrow cleared	Return successful
borrowThenReturn_fullFlow_updatesAvailability_andBorrowRecordStatus()	Ensures full borrow-return lifecycle works	Record marked returned	Flow completed
borrowFlow_memberNotFound_throwsRuntimeException()	Prevents borrowing with invalid member	Exception thrown	Exception thrown
borrowFlow_itemNotFound_throwsRuntimeException()	Prevents borrowing invalid item	Exception thrown	Exception thrown
borrowFlow_itemNotAvailable_throwsRuntimeException()	Prevents borrowing unavailable item	Exception thrown	Exception thrown
borrowFlow_inactiveMember_throwsRuntimeException()	Prevents inactive members from borrowing	Exception thrown	Exception thrown

borrowFlow_borrowLimitReached_throwsRuntimeException()	Enforces borrowing limits	Exception thrown	Exception thrown
borrowFlow_memberHasOverdue_throwsRuntimeException()	Blocks borrowing with overdue items	Exception thrown	Exception thrown
returnFlow_notOverdue_doesNotSendAlert()	Ensures no alert for on-time return	No alert sent	No alert sent
returnFlow_overdue_sendsAlertToCorrectMemberWithMessage()	Sends alert for overdue return	Alert sent with message	Alert sent
returnFlow_noActiveBorrow_throwsRuntimeException()	Prevents returning non-borrowed item	Exception thrown	Exception thrown

Part 3: System Testing

Project Context

This section presents the **system testing activities** carried out for the **Smart Library System**. System testing is performed to validate the behavior of the **entire system as a whole**, ensuring that all components work together correctly through the system's public interface.

Purpose of System Testing

The main goals of system testing in this project are to:

- Verify that the complete system satisfies the specified functional requirements
- Validate end-to-end user workflows and real usage scenarios
- Ensure correct interaction across all system layers
- Confirm that business rules are enforced consistently at system level

System Testing Approach

System testing for the Smart Library System was performed by:

- Testing the system **end-to-end through the LibraryFacade**, which represents the system's public entry point
- Using **real implementations** of all components (storages, policies, services, and notifications)
- Avoiding mocks and stubs to reflect real system behavior
- Simulating realistic user actions such as borrowing, returning, and capacity enforcement

This approach ensures that the system behaves correctly from the user's perspective.

Test Organization and Responsibilities

- System tests are organized around **complete user scenarios**
- Each test validates a specific system-level behavior or rule
- Team members were assigned 4 system test cases each, based on scenario complexity
- Each responsible member implemented their assigned tests and documented the results in the table below

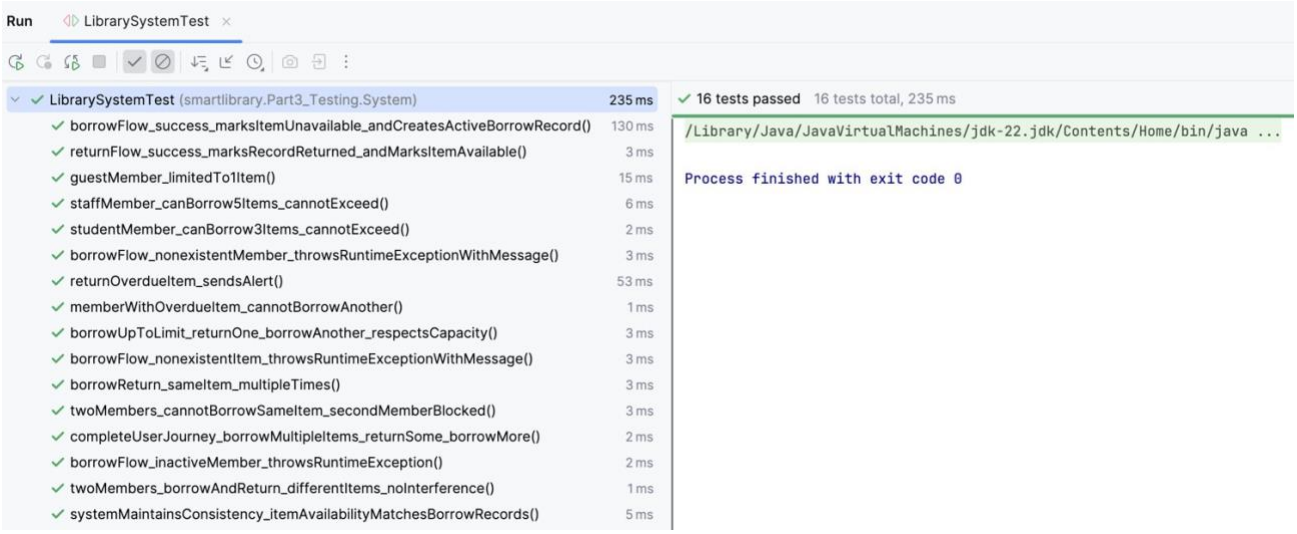
System Testing Documentation Table

Class: LibrarySystemTest
Responsible Members: All Group

Test Case	Test Description & Reason	Expected Output	Actual Result
Arsildo Veliu			
borrowFlow_success_marksItemUnavailable_andCreatesActiveBorrowRecord()	Verifies that a successful borrow operation marks the item as unavailable and creates an active borrow record in the system	Item marked unavailable, active borrow record created	Item unavailable, borrow record created
returnFlow_success_marksRecordReturned_andMarksItemAvailable()	Verifies that a successful return operation marks the borrow record as returned and restores item availability	Item marked available, borrow record marked returned	Item available, record returned
borrowFlow_nonexistentMember_throwsRuntimeExceptionWithMessage()	Ensures the system prevents borrowing when member does not exist and provides clear error message	RuntimeException with "Member not found" message	Exception thrown with correct message
borrowFlow_nonexistentItem_throwsRuntimeExceptionWithMessage()	Ensures the system prevents borrowing when item does not exist and provides clear error message	RuntimeException with "Item not found" message	Exception thrown with correct message
Megi Almadhi			
completeUserJourney_borrowMultipleItems_returnSome_borrowMore()	Tests a realistic user workflow: borrow multiple items, return some, borrow additional items. Verifies capacity tracking across multiple operations	Active borrows correctly tracked through lifecycle: 0→2→1→2	Borrow count changes correctly through all operations

studentMember_canBorrow3Items_cannotExceed()	Verifies that STUDENT members can borrow up to their limit (3) and are prevented from exceeding it	3 items borrowed successfully, 4th item rejected with policy violation	Limit enforced correctly
borrowUpToLimit_returnOne_borrowAnother_respectsCapacity()	Tests capacity management: GUEST borrows to limit (1), cannot borrow more, returns item, then can borrow different item	Borrow blocked at capacity, allowed after return	Capacity respected throughout workflow
borrowReturn_sameItem_multipleTimes()	Tests that the same item can be borrowed and returned multiple times, creating separate borrow records	3 separate borrow records created, all marked returned	Multiple borrow-return cycles work correctly
Sidrit Rrustemi			
staffMember_canBorrow5Items_cannotExceed()	Verifies that STAFF members can borrow up to their limit (5) and are prevented from exceeding it	5 items borrowed successfully, 6th item rejected with policy violation	Limit enforced correctly
guestMember_limitedTo1Item()	Verifies that GUEST members can borrow up to their limit (1) and are prevented from exceeding it	1 item borrowed successfully, 2nd item rejected with policy violation	Limit enforced correctly
twoMembers_cannotBorrowSameItem_secondMemberBlocked()	Tests item contention: when one member borrows an item, another member (even with higher privileges) cannot borrow it until returned	Second borrow blocked, succeeds after return	Item availability prevents concurrent borrows
borrowFlow_inactiveMember_throwsRuntimeException()	Ensures inactive members are prevented from borrowing items	RuntimeException with "policy violation" message	Inactive status blocks borrowing

Viola Zeneli			
twoMembers_borrowAndReturn_differentItems_noInterference()	Verifies that multiple members can borrow different items simultaneously without interfering with each other's operations	Member 1 returns item, Member 2's borrow unaffected	Members operate independently
memberWithOverdueItem_cannotBorrowAnother()	Tests that members with overdue items are blocked from borrowing additional items until overdue items are returned. NOTE: Creates overdue state manually due to time dependency limitation	Borrow blocked with "policy violation" when overdue exists	Overdue status prevents new borrows
returnOverdueItem_sendsAlert()	Verifies that returning an overdue item triggers an alert to the member with fine information. NOTE: Creates overdue state manually due to time dependency limitation	Alert sent to member with overdue message and fine	Alert service called with correct member ID and message
systemMaintainsConsistency_itemAvailabilityMatchesBorrowRecords()	Verifies data consistency across the system: item availability status matches borrow records state	Item availability consistent with active borrow records	System maintains consistency between storage layers



Part 1: GitHub Usage

This section provides a **detailed and systematic analysis of GitHub usage and commit activity** for the group project. The purpose of this analysis is to demonstrate **how consistently and collaboratively GitHub was used by all group members**, not merely whether commits exist.

The evaluation is based **directly on the provided GitHub commit history screenshots**, which will be included as visual evidence for verification. All observations are derived from commit dates, commit messages, authorship, and the explicit labeling of individual versus **all-group** contributions.

The analysis focuses on the following criteria:

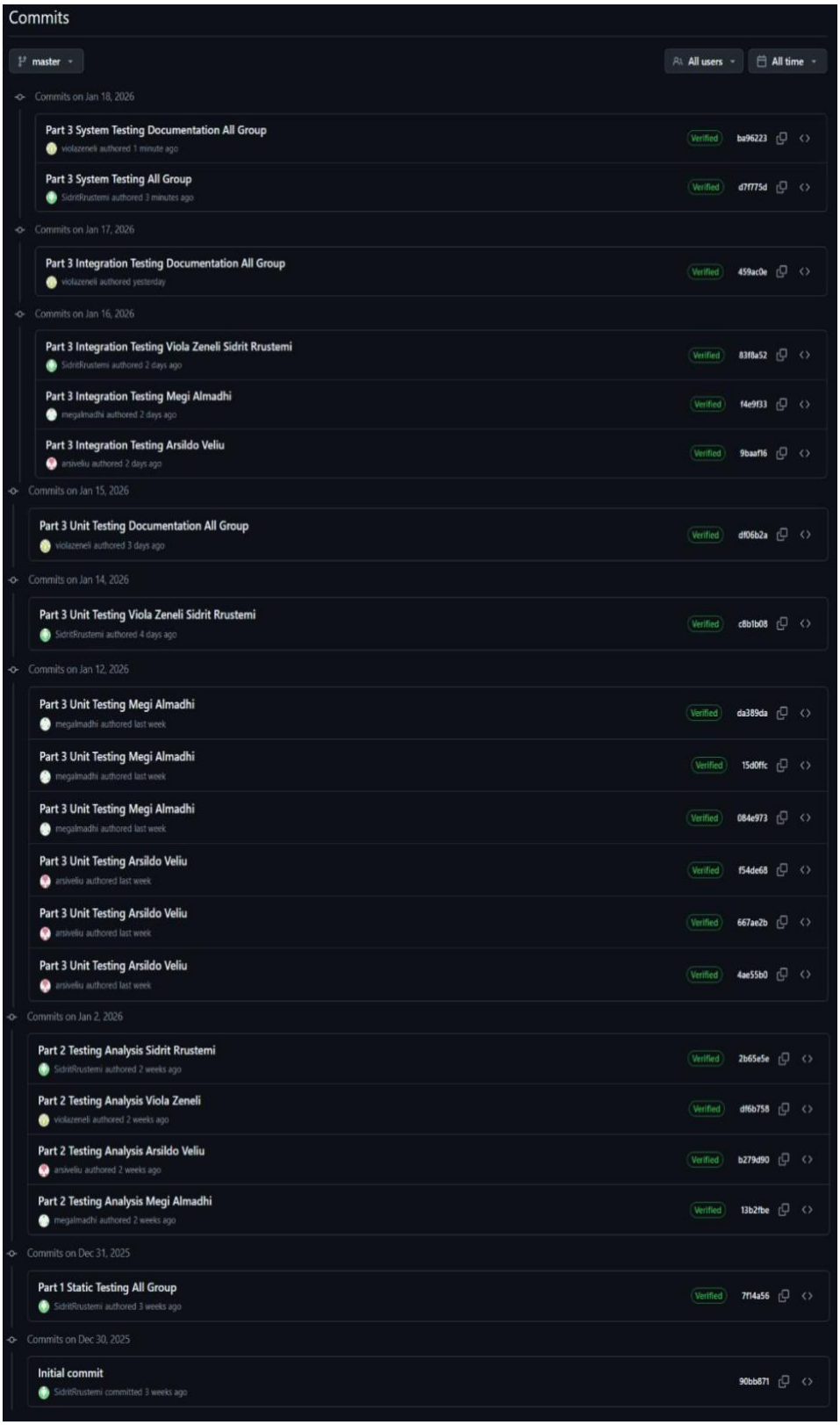
- Frequency and timing of commits across the project duration
- Clear separation between individual responsibilities and **all-group work**
- Progressive development aligned with project phases
- Evidence of continuous collaboration rather than last-minute submissions

Project Commit Timeline

- **Dec 30, 2025** – Initial repository setup
- **Dec 31, 2025** – Part 1: Static Testing (**All Group**)
- **Jan 2, 2026** – Part 2: Testing Analysis (Individual submissions)
- **Jan 12–15, 2026** – Part 3: Unit Testing (Individual work + All Group documentation)
- **Jan 16–17, 2026** – Part 3: Integration Testing (Individual work + All Group documentation)
- **Jan 18, 2026** – Part 3: System Testing (**All Group work and documentation**)

This timeline shows a **clear, phased, and systematic progression** of the project using GitHub.

GitHub Commit History



Individual Commit Documentation

Sidrit Rrustemi

Documented commits:

- Initial commit establishing the repository structure (Dec 30, 2025)
- Part 1 Static Testing – **All Group contribution** (Dec 31, 2025)
- Part 2 Testing Analysis – Individual contribution (Jan 2, 2026)
- Part 3 Unit Testing – Joint work with Viola Zeneli (Jan 14, 2026)
- Part 3 Integration Testing – Joint work with Viola Zeneli (Jan 16, 2026)
- Part 3 System Testing – **All Group contribution** (Jan 18, 2026)

Systematic usage explanation: Sidrit Rrustemi contributed consistently from the start of the project until the final phase. His commits appear in the repository setup, individual analytical work, collaborative testing phases, and final system testing. The timing and labeling of commits reflect structured participation aligned with the project schedule.

Viola Zeneli

Documented commits:

- Part 2 Testing Analysis – Individual contribution (Jan 2, 2026)
- Part 3 Unit Testing Documentation – **All Group documentation** (Jan 15, 2026)
- Part 3 Integration Testing Documentation – **All Group documentation** (Jan 17, 2026)
- Part 3 System Testing Documentation – **All Group documentation** (Jan 18, 2026)
- Joint Unit and Integration Testing commits with Sidrit Rustemi

Systematic usage explanation: Viola Zeneli contributed both individual analytical work and multiple **all-group documentation submissions**. Her commits are distributed across different testing phases, demonstrating a structured role in documenting and consolidating group work. The repetition of documentation commits across phases confirms continuous involvement rather than isolated activity.

Megi Almadhi

Documented commits:

- Part 2 Testing Analysis – Individual contribution (Jan 2, 2026)
- Part 3 Unit Testing – Multiple individual commits (Jan 12, 2026)
- Part 3 Integration Testing – Individual contribution (Jan 16, 2026)

Systematic usage explanation: Megi Almadhi's GitHub activity shows an iterative development approach, with multiple commits during the Unit Testing phase followed by a separate Integration Testing submission. This pattern indicates step-by-step refinement rather than single bulk commits, which reflects systematic use of version control.

Arsildo Veliu

Documented commits:

- Part 2 Testing Analysis – Individual contribution (Jan 2, 2026)
- Part 3 Unit Testing – Multiple individual commits (Jan 12, 2026)
- Part 3 Integration Testing – Individual contribution (Jan 16, 2026)

Systematic usage explanation: Arsildo Veliu contributed across multiple testing phases with clearly separated commits. Similar to other members, multiple Unit Testing commits indicate incremental progress. The commits follow the planned project structure and are not concentrated in a single date.

All-Group Contributions

The following commits clearly indicate **collective group work**:

- Part 1 Static Testing – All Group
- Part 3 Unit Testing Documentation – All Group
- Part 3 Integration Testing Documentation – All Group
- Part 3 System Testing – All Group
- Part 3 System Testing Documentation – All Group

These commits confirm that **all members collaborated on shared deliverables**, not only on individual tasks.

Overall GitHub Usage Assessment

Based on the documented commit history:

- All members contributed individual work
- All members participated in **group-based tasks and documentation**
- Commits are distributed across time and project phases
- Commit messages are clear, descriptive, and phase-specific

The repository demonstrates **systematic, transparent, and well-coordinated GitHub usage**, appropriate for an academic group project and consistent with good version control practices.

Conclusion

The commit history provides clear evidence that the project was developed collaboratively and systematically using GitHub. Individual responsibilities and **all-group contributions** are both visible and well-documented. The structured timeline and consistent commit behavior support the conclusion that all group members actively participated throughout the entire project lifecycle.